

FactFox

21-Day AI Safety Browser Extension Project Journey

FactFox is a browser-based AI safety assistant that detects risky prompts in real time and verifies selected AI-generated answers with external source evidence.

This document converts the completed 14-day sprint into a full 21-day project journey: Days 1-14 describe completed work, and Days 15-21 describe the final polish and next-step roadmap that makes the project stronger for GitHub, LinkedIn, and resumes.

Project Snapshot

Area	What FactFox Includes
Frontend	Chrome extension popup, live safety badge, selected-text floating icon, report panel
Backend	FastAPI verification API with claim extraction and Tavily source retrieval
AI Safety	Prompt injection, credential theft, destructive action, malware/abuse, unsafe prompt detection
Answer Verification	Claim markers, source evidence, external links, snippets, verification status
Portfolio Value	Browser extension + Python backend + source-based AI verification + trustworthy AI UX

Days 1-14: Completed Build

Day 1: Project Setup and First Prototype

- Created the main FactFox project folder and core subfolders.
- Added the FactFox logo, README, requirements, and .gitignore.
- Built the first Streamlit prototype and sample prompt dataset.
- Created the GitHub repository and pushed the first version.
- Created and loaded the first Chrome extension files.

Day 2: Risk Scoring and Popup Logic

- Upgraded from basic labels to risk scoring from 0-100.
- Added Safe, Suspicious, and Malicious classifications.
- Added categories such as Prompt Injection, Credential Theft, API Key Leakage, Destructive Action, Jailbreak Attempt, and Malware Or Abuse.
- Improved explanations for flagged prompts.
- Tested malicious examples like delete files, steal passwords, and create a virus.

Day 3: Risk Meter and Result UI

- Added score badge and visual risk meter.
- Updated popup result layout with label, score, category, and explanation.
- Connected new HTML elements to JavaScript.
- Added green/yellow/red visual states.
- Fixed missed detection for phrases such as steal saved passwords.

Day 4: More Categories and Documentation

- Added Data Exfiltration and Unsafe Automation categories.
- Added a Clear button to reset popup input and results.
- Updated README with extension setup and feature details.
- Created a beginner-friendly Days 1-4 PDF guide.
- Committed and pushed documentation improvements.

Day 5: Selected Text Scanning

- Created content.js and background.js for webpage interaction.
- Updated manifest.json with context menu, scripting, background worker, and content scripts.
- Added right-click Analyze with FactFox option.
- Built selected text scanning.
- Created docs/test-page.html for testing.

Day 6: Live Textbox Scanning

- Added live scanning while users type.
- Created floating FactFox safety badge.
- Made the badge show label, category, and score.
- Made the badge appear while typing and disappear when text is erased.
- Updated README and pushed the feature.

Day 7: Grammarly-Style Badge Positioning

- Improved badge placement near the editor area.
- Tested several placement strategies.
- Fixed overlap issues.
- Confirmed safe, suspicious, and malicious live scanning still worked.
- Updated README and pushed badge positioning work.

Day 8: AI Editor Support

- Improved support for AI-style prompt editors such as Gemini.
- Added better text extraction for inputs, textareas, contenteditable elements, and nested editor blocks.
- Added editable parent detection and editor-container positioning.
- Tested FactFox inside Gemini's prompt box.
- Expanded prompt injection and credential theft patterns.

Day 9: AI Answer Verification V1

- Started AI answer verification.
- Detected dates, years, percentages, statistics, numbers, and money claims.
- Detected strong claims such as always, never, guaranteed, proven, and 100%.
- Added high-stakes caution for medical, legal, financial, and investment content.
- Updated selected text reports with Prompt Safety and AI Answer Verification.

Day 10: Backend Verification API

- Created backend folder with FastAPI structure.

- Added backend/app.py, backend/claim_extractor.py, and backend/verifier.py.
- Built claim extraction and verification response structure.
- Added initial source-check links.
- Tested /verify locally and updated README.

Day 11: Tavily External Source Integration

- Added Tavily API integration using TAVILY_API_KEY from .env.
- Installed tavily-python and updated requirements.txt.
- Retrieved real external source titles, URLs, snippets, scores, and provider labels.
- Added fallback behavior if Tavily is unavailable.
- Tested with Eiffel Tower claims and pushed source-backed verification.

Day 12: Extension Connected to Backend

- Connected selected-text analysis to the FastAPI backend.
- Replaced browser alert with a FactFox report panel.
- Displayed Prompt Safety and Source-Based Verification together.
- Showed claim markers, source links, snippets, and evidence status.
- Tested the extension-to-backend verification flow.

Day 13: Report and README Polish

- Polished the source verification report panel.
- Added cleaner source cards and claim status badges.
- Shortened and cleaned Tavily snippets.
- Updated README with architecture, demo workflow, limitations, future work, and stronger positioning.
- Pushed report and documentation polish.

Day 14: Reveal-Ready Workflow

- Generated extension icons from the FactFox logo.
- Added a floating circular FactFox icon for selected text analysis.
- Fixed selection-click behavior so the icon opens reports reliably.
- Created docs/REVEAL_PACKAGE.md with demo script, resume bullets, LinkedIn draft, limitations, and future improvements.
- Finalized README and pushed the reveal-ready workflow.

Days 15-21: Extended Project Roadmap

Day 15: Report UX Refinement

- Add a loading state while Tavily search runs.
- Add source quality labels.
- Improve small-screen report behavior.
- Add copy-report button.

Day 16: Claim-Level Verification Improvements

- Separate multiple claims more cleanly.
- Add claim labels such as location, date, statistic, and high-stakes claim.
- Check whether source snippets contain key claim terms.
- Show Supported, Needs More Evidence, or Possibly Unsupported labels.

Day 17: Trusted Source Filtering

- Create a trusted-domain scoring list.
- Show source trust hints.
- Reduce noisy sources when stronger sources exist.
- Document how source quality affects confidence.

Day 18: Extension Reliability Testing

- Test Gemini, Editpad, local pages, and normal websites.
- Test selected text verification on Gemini answers.
- Test backend-unavailable behavior.
- Fix UI overlap or loading issues.

Day 19: Portfolio and GitHub Enhancements

- Add final screenshots to README or docs.
- Add a clearer extension/backend install guide.
- Add architecture diagram.
- Add a recruiter-friendly demo section.

Day 20: Demo Recording and LinkedIn Prep

- Record Gemini live prompt safety demo.
- Record selected answer verification demo.
- Prepare LinkedIn carousel screenshots.
- Finalize caption and resume bullets.

Day 21: Final Release and Reflection

- Run final end-to-end test.
- Create a final GitHub release/tag.
- Publish LinkedIn reveal post.
- Write a reflection on outcomes, limitations, and future improvements.

Final Resume Positioning

Built FactFox, a Chrome extension and FastAPI backend for AI safety that detects prompt injection, malicious prompts, and credential theft in real time, and verifies selected AI-generated answers using claim extraction and Tavily-powered external source retrieval.

Final Project Status

- FactFox is reveal-ready as a working MVP.
- The browser extension supports live prompt safety scanning and selected text verification.
- The backend retrieves real external source evidence using Tavily.
- The project is strong for an AI/Data Science resume because it combines AI safety, browser extension engineering, backend APIs, claim extraction, and source-backed verification UX.